

Neptune's War

Documentación Técnica · Fase 2 (Informe Final)

Servidor Colyseus · Túnel ngrok · Cliente Unity

Juego de Batalla Naval Multijugador en Red

Abril 2026

Equipo de Desarrollo Neptune's War

Índice de Contenidos

1. Descripción General de la Fase 2
2. Sistema de Conexión en Red Implementado
 - 2
 - .
 - 1 Arquitectura general Cliente-Servidor
 - 2
 - .
 - 2 Servidor Colyseus (Node.js + TypeScript)
 - 2
 - .
 - 3 Túnel ngrok
 - 2
 - .
 - 4 Protocolo WebSocket y sincronización de estado
 - 2
 - .
 - 5 Flujo completo de una partida
3. Descripción de Pruebas Realizadas y Resultados
 - 3
 - .
 - 1 Prueba de conexión y sala (onJoin / onLeave)
 - 3
 - .
 - 2 Prueba de fase de colocación
 - 3
 - .
 - 3 Prueba de fase de batalla
 - 3
 - .
 - 4 Prueba de victoria y fase result
 - 3
 - .
 - 5 Prueba de desconexión de un jugador
 - 3
 - .
 - 6 Resumen de resultados
4. Mejoras y Posibles Optimizaciones Futuras
 - 4
 - .
 - 1 Seguridad y autenticación

4

.

2 Escalabilidad del servidor

4

.

3 Calidad del código y mantenibilidad

4

.

4 Experiencia de usuario y robustez

4

.

5 Infraestructura y despliegue

5. Conclusiones

1. Descripción General de la Fase 2

La **Fase 2** de Neptune's War consistió en la implementación completa del **servidor de juego** que permite la comunicación en red entre dos clientes Unity. Mientras que la Fase 1 sentó las bases del cliente (lógica local, esquemas de datos, controladores de escena y plan de integración de red), la Fase 2 pone en marcha el lado servidor del sistema y realiza la integración real cliente ↔ servidor.

Los objetivos de esta fase fueron:

- Crear el servidor **Colyseus** en Node.js/TypeScript con la sala *GameRoom*.
- Definir el estado compartido (*State*, *Player*) usando *@colyseus/schema*.
- Implementar toda la lógica de negocio del juego en el servidor: colocación de barcos, turnos, detección de victoria.
- Exponer el servidor mediante **ngrok** para que el cliente Unity pueda conectarse desde cualquier red.
- Verificar el funcionamiento end-to-end mediante pruebas manuales y de integración.

■■ Repositorio: **Jesus721MLG/server** · Lenguaje: TypeScript (Node.js) · Framework: Colyseus v0.10.x · Puerto: 2567 · Túnel: ngrok (dominio estático)

2. Sistema de Conexión en Red Implementado

2.1 Arquitectura general Cliente-Servidor

El sistema sigue el patrón **Cliente-Servidor con sincronización de estado reactiva**. El servidor es la única fuente de verdad (*source of truth*): toda la lógica de juego (asignación de asientos, fases, disparos, victoria) se ejecuta en el servidor y el estado resultante se difunde automáticamente a todos los clientes.

```
[Unity Client A] [Unity Client B]
| |
| WebSocket (wss://) |
+-----+-----+
|
| [ngrok Tunnel]
| <-- expone puerto 2567 -->
|
| [Colyseus Server] (Node.js)
|   __ Room 'game' (GameRoom)
|   __ State (phase, playerTurn, winningPlayer,
|   | players, player1Shots, player2Shots)
|   __ Lógica de turnos y validación
```

Figura 1 – Diagrama de la arquitectura del sistema

2.2 Servidor Colyseus (Node.js + TypeScript)

El servidor está compuesto por tres archivos principales:

index.ts – Punto de entrada

Crea el servidor HTTP con *express*, instancia el servidor Colyseus y registra la sala game. Opcionalmente monta el panel de monitoreo en */colyseus*. Escucha en el puerto **2567** (configurable mediante la variable de entorno PORT).

```
const gameServer = new Server({ server: createServer(app) });
gameServer.register('game', GameRoom);
app.use('/colyseus', monitor(gameServer));
gameServer.listen(port);
```

src/state.ts – Esquemas de estado

Define las clases **Player** y **State** como subclases de *@colyseus/schema*. El decorador *@type* declara cada propiedad como sincronizable, lo que permite a Colyseus generar parches binarios mínimos (delta) y enviarlos a los clientes sólo cuando cambian.

Clase	Campo	Tipo	Descripción
Player	sessionId	string	ID único de sesión WebSocket
Player	seat	int16	Número de jugador (1 ó 2)
State	phase	string	Fase actual: waiting/place/battle/result

State	playerTurn	int16	Jugador activo en el turno actual
State	winningPlayer	int16	Ganador (-1 = sin ganador)
State	players	MapSchema<Player>	Mapa sessionId → Player
State	player1Shots	ArraySchema<int16>	64 celdas de disparos del jugador 1
State	player2Shots	ArraySchema<int16>	64 celdas de disparos del jugador 2

src/game-room.ts – Lógica del juego

La clase **GameRoom** extiende `Room<State>` y centraliza toda la lógica de negocio. Sus propiedades internas son:

- **maxClients = 2** – la sala sólo admite exactamente dos jugadores.
- **gridSize = 8** – cuadrícula de 8×8 = 64 celdas.
- **startingFleetHealth = 10** – suma de celdas de barcos: Scout(2)+Cruiser(3)+Carrier(5).
- **playerHealth[2]** – puntos de vida de cada jugador (inicia en 10).
- **placements[2][64]** – cuadrículas de colocación de cada jugador.
- **playersPlaced** – contador para saber cuándo ambos jugadores han colocado.

Los métodos del ciclo de vida de la sala son:

Método	Responsabilidad
<code>onInit()</code>	Inicializa el estado limpio llamando a <code>reset()</code> .
<code>onJoin(client)</code>	Crea un <code>Player</code> , le asigna <code>seat</code> (1 ó 2) y lo añade a <code>state.players</code> . Al llegar el 2.º jugador cambia la fase a 'place' y bloquea la sala (<code>lock()</code>).
<code>onLeave(client)</code>	Elimina al jugador del mapa, decrementa el contador y resetea la fase a 'waiting' para que el rival sea notificado.
<code>onMessage(client, msg)</code>	Despacha los comandos 'place' y 'turn' con validación completa (ver §2.4).
<code>onDispose()</code>	Libera recursos al destruir la sala.
<code>reset()</code>	Reinicia salud, placements y estado compartido; inicializa los 64 elementos de los arrays de disparos a 0.

2.3 Túnel ngrok

Durante el desarrollo, el servidor corre localmente en localhost:2567. Para que el cliente Unity —que puede estar en otra máquina o red— se conecte, se utiliza **ngrok** como intermediario.

- ngrok abre un túnel seguro TLS entre Internet y el proceso local.
- Se usa un **dominio estático** de ngrok (*.ngrok-free.dev) vinculado a la cuenta, por lo que la URL **no cambia** al reiniciar el túnel.
- El cliente Unity tiene la URL hardcodeada: wss://shrivel-stretch-doorman.ngrok-free.dev

```
# Arrancar el túnel
ngrok http 2567 --domain=shrivel-stretch-doorman.ngrok-free.dev
```

■■ La URL estática de ngrok es suficiente para desarrollo/prototipado. Para producción se recomienda un servidor con IP pública propia (VPS) y un certificado TLS propio, eliminando la dependencia de ngrok.

2.4 Protocolo WebSocket y sincronización de estado

La comunicación se realiza íntegramente sobre **WebSocket** (WSS en producción). Colyseus implementa un protocolo binario propio sobre WebSocket que incluye:

- **Delta patches** – sólo se envían los campos del estado que han cambiado, minimizando el ancho de banda.
- **Serialización binaria** – @colyseus/schema serializa el estado en binario compacto, más eficiente que JSON.
- **Mensajes de sala** – mensajes JSON personalizados del cliente al servidor (comandos place y turn).

Mensajes definidos entre cliente y servidor:

Dirección	Comando	Payload	Descripción
Cliente → Servidor	place	{ command, placement: int[] }	Array de 64 valores (0=vacío, >0=barco) con la disposición del jugador.
Cliente → Servidor	turn	{ command, targetIndex: int }	Índice lineal (0–63) de la celda atacada.
Servidor → Cliente	(delta state)	Binario @schema	Cambios de fase, turno, disparos, jugador ganador.

2.5 Flujo completo de una partida

A continuación se describe el ciclo de vida completo de una partida:

1	waiting	El primer cliente se conecta y entra en la sala. El servidor asigna seat=1. La fase permanece en waiting .
---	---------	---

2	waiting→place	El segundo cliente se conecta: seat=2. El servidor llama a lock() y cambia la fase a place . Ambos clientes reciben el delta.
3	place	Cada cliente envía su comando place con el array de 64 celdas. El servidor almacena el placement en placements[seat-1]. Cuando playersPlaced == 2, la fase cambia a battle .
4	battle	El jugador con playerTurn == seat envía turn con targetIndex. El servidor valida el turno, registra hit (1) o miss (2) en el array correspondiente y actualiza playerHealth.
5	battle	Tras cada disparo, si la salud del oponente llega a 0, la fase cambia a result y se fija winningPlayer. Si no, se alterna playerTurn.
6	result	Ambos clientes reciben el estado final y muestran el ganador en la UI.
7	leave	Si un jugador abandona en cualquier fase, el servidor resetea la fase a waiting y desbloquea la sala para aceptar un nuevo jugador.

3. Descripción de Pruebas Realizadas y Resultados

Las pruebas se realizaron de forma **manual e integrada**, conectando dos instancias del cliente Unity al servidor Colyseus a través del túnel ngrok. Se verificó cada transición de fase, el intercambio de mensajes y el comportamiento ante condiciones límite. A continuación se describen las pruebas individuales y sus resultados.

3.1 Prueba de conexión y sala

Prueba 3.1: Conexión y asignación de asientos	PASS
Objetivo: Verificar que dos clientes se conectan, obtienen asientos 1 y 2, y la fase cambia a 'place'.	
Procedimiento: Se arrancó el servidor (npm start) y el túnel ngrok. Se abrieron dos instancias de Unity apuntando a la URL ngrok. Se monitorizaron los logs del servidor.	
Resultado: Ambos clientes aparecieron en el panel /colyseus. Los logs mostraron ' <i>client joined</i> ' con sessionId distintos. El estado pasó de waiting a place al conectarse el segundo cliente. Cada cliente recibió su seat correctamente (1 y 2).	

3.2 Prueba de fase de colocación

Prueba 3.2: Envío de placement y transición a battle	PASS
Objetivo: Verificar que al enviar ambos placements el servidor inicia la fase de batalla.	
Procedimiento: Desde Unity, los dos jugadores colocaron sus barcos (Scout 2 celdas, Cruiser 3, Carrier 5) y enviaron el comando place. Se inspeccionó el estado con el monitor Colyseus.	
Resultado: El servidor recibió ambos comandos; tras el segundo, playersPlaced llegó a 2 y la fase cambió a battle . Ambos clientes recibieron el delta y la UI de Unity cargó el modo de ataque. Los arrays placements internos contenían los valores correctos.	

3.3 Prueba de fase de batalla

Prueba 3.3: Turnos y registro de disparos (hit y miss)	PASS
Objetivo: Verificar que el servidor respeta el turno activo, registra impactos y fallos correctamente y alterna el turno.	
Procedimiento: El jugador 1 atacó una celda vacía (miss) y luego una celda con barco (hit). Se intentó disparar fuera de turno desde el jugador 2 para probar la validación. Se inspeccionaron player1Shots y player2Shots en el monitor.	
Resultado: El disparo fuera de turno fue ignorado (if playerTurn != seat return). Los disparos válidos actualizaron el array: 0→2 (miss) y 0→1 (hit). playerHealth[1] decrementó en 1 tras el hit. El turno alternó a jugador 2 correctamente.	

3.4 Prueba de victoria

Prueba 3.4: Detección de victoria y fase result	PASS
Objetivo: Verificar que al eliminar todas las celdas de barco del rival la fase cambia a 'result' y se registra el ganador.	
Procedimiento: Se simuló una partida hasta que el jugador 1 impactó las 10 celdas de barco del jugador 2 (health[1] → 0). Se verificó el estado en el monitor y la UI de Unity.	
Resultado: playerHealth[1] llegó a 0; el servidor asignó state.winningPlayer = 1 y cambió la fase a result . Ambos clientes recibieron el estado final y Unity mostró el mensaje de victoria/derrota correspondiente.	

3.5 Prueba de desconexión

Prueba 3.5: Desconexión de un jugador durante la partida	PASS
Objetivo: Verificar que al desconectarse un jugador el servidor notifica al rival y resetea la fase.	
Procedimiento: Durante la fase de batalla se cerró la instancia Unity de uno de los jugadores. Se monitorizaron los logs del servidor y el estado del cliente restante.	
Resultado: El servidor ejecutó onLeave, eliminó al jugador del mapa y cambió la fase a waiting . El cliente restante recibió el delta y ConnectingSceneController cargó la escena de conexión. No se produjeron errores ni excepciones en el servidor.	

3.6 Resumen de resultados

Prueba	Área	Resultado
3.1 Conexión y asientos	Red / Sala	■ PASS
3.2 Placement → Battle	Protocolo / Estado	■ PASS
3.3 Turnos y disparos	Lógica de juego	■ PASS
3.4 Victoria / Result	Lógica de juego	■ PASS
3.5 Desconexión	Resiliencia	■ PASS

4. Mejoras y Posibles Optimizaciones Futuras

El sistema funciona correctamente para el contexto de prototipo/desarrollo para el que fue diseñado. Sin embargo, para llevar Neptune's War a un entorno de producción robusto, escalable y seguro, se identifican las siguientes áreas de mejora:

4.1 Seguridad y Autenticación

URL del servidor como variable de configuración

Actualmente la URL WSS está hardcodeda en GameClient.cs. Se recomienda moverla a un **ScriptableObject** de Unity o a un archivo de configuración externo, permitiendo cambiarla sin recompilar el cliente.

Validación de placement en el servidor

El servidor acepta cualquier array de 64 enteros como placement sin verificar que la distribución de barcos sea válida (tamaños correctos, sin solapamientos, dentro del tablero). Se recomienda añadir validación server-side para prevenir trampas.

```
// Validación sugerida en onMessage 'place'
if (!isValidPlacement(message.placement, gridSize)) return;
// Verificar: exactamente 2+3+5 celdas ocupadas, sin barcos discontinuos
```

Autenticación de jugadores

No existe ningún mecanismo de autenticación. Cualquier WebSocket que llegue al servidor puede unirse a la sala. Para producción se recomienda:

- Tokens JWT generados por un servicio de autenticación externo.
- Verificación del token en onAuth() de Colyseus antes de permitir el onJoin.

4.2 Escalabilidad del Servidor

Actualización de versiones

El servidor usa **Colyseus v0.10.7** (2018) y **Node.js 8.9.1**, versiones ya fuera de soporte. Migrar a Colyseus **0.15.x** o superior y Node.js **18 LTS** aportaría mejoras de rendimiento, nuevas API y parches de seguridad. La nueva API de Colyseus reemplaza onInit/onMessage(client, msg) por onCreate/onMessage(client, type, msg).

Salas múltiples y matchmaking

Actualmente existe una sola sala game. Para soportar múltiples partidas simultáneas, Colyseus crea instancias de sala automáticamente, pero se recomienda implementar un sistema de **matchmaking** con colas de espera.

Persistencia de estado

El estado de la partida reside únicamente en memoria RAM. Si el servidor se reinicia, todas las partidas en curso se pierden. Para mayor resiliencia se puede integrar una base de datos ligera (**Redis** para estado efímero, **PostgreSQL** para estadísticas y historial).

4.3 Calidad del Código y Mantenibilidad

Configuración por entorno

Los valores de `gridSize`, `startingFleetHealth` y `maxClients` están hardcodedos en `GameRoom`. Se recomienda extraerlos a un archivo de configuración (p. ej. `config.ts`) o a variables de entorno para facilitar el ajuste sin modificar código fuente.

Cobertura de tests automáticos

No existen tests unitarios ni de integración automáticos. Se recomienda añadir:

- **Tests unitarios** de la lógica de `GameRoom` con *jest* o *mocha*.
- **Tests de integración** usando el cliente `colyseus.js` directamente contra el servidor en memoria.
- **CI/CD**: añadir un workflow de GitHub Actions que ejecute los tests en cada push.

Tipado más estricto

El método `onMessage(client, message)` acepta `message: any`. Definir interfaces TypeScript para cada comando (`PlaceCommand`, `TurnCommand`) eliminaría errores de acceso a propiedades indefinidas.

4.4 Experiencia de Usuario y Robustez

Reinicio de sala tras resultado

Cuando la fase llega a **result**, la sala no se reinicia automáticamente; los jugadores deben reconectarse. Se podría añadir un comando `rematch` que, si ambos jugadores lo envían, llame a `reset()` y vuelva a la fase **place** sin desconexión.

Gestión de reconexión

Actualmente una desconexión temporal (pérdida de Wi-Fi momentánea) equivale a una salida permanente. `Colyseus 0.14+` incluye `allowReconnection(client, seconds)` para reservar el asiento durante un tiempo y permitir la reconexión transparente.

Validación de celdas repetidas

El servidor actualmente ignora los disparos a celdas ya atacadas (`if shots[targetIndex] == 0`), pero no notifica al cliente. Sería útil enviar un mensaje de error explícito para que la UI informe al jugador.

4.5 Infraestructura y Despliegue

Eliminar dependencia de ngrok en producción

`ngrok` es ideal para desarrollo y demos, pero introduce latencia adicional y depende de un servicio externo. Para producción se recomienda desplegar el servidor en un **VPS** (AWS EC2, DigitalOcean, Fly.io) con IP pública y certificado TLS propio.

Contenedor Docker

Crear un Dockerfile y un docker-compose.yml para el servidor facilitaría el despliegue reproducible en cualquier entorno y simplificaría la integración con plataformas de orquestación como Kubernetes.

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build
EXPOSE 2567
CMD ["node", "dist/index.js"]
```

Monitor de producción

El panel /colyseus no tiene protección por contraseña y expone información sensible de la sala. En producción se debe proteger con middleware de autenticación o desactivarlo completamente.

5. Conclusiones

La **Fase 2 de Neptune's War** ha concluido con la implementación exitosa del servidor Colyseus y su integración con el cliente Unity desarrollado en la Fase 1. El sistema cumple todos los objetivos planteados:

- ■ Dos jugadores pueden conectarse desde redes distintas a través de ngrok.
- ■ El servidor gestiona correctamente el ciclo de vida completo de una partida.
- ■ La sincronización de estado es reactiva y eficiente gracias a @colyseus/schema.
- ■ La lógica de negocio (turnos, hits, victoria) reside íntegramente en el servidor.
- ■ Las desconexiones se manejan sin provocar errores en el servidor ni en el cliente.

El mayor logro técnico de esta fase es la **integración completa del protocolo cliente-servidor**: el esquema de estado definido en TypeScript (state.ts) se sincroniza automáticamente con las clases C# del cliente Unity, eliminando la necesidad de serialización/deserialización manual y garantizando consistencia entre ambos extremos de la conexión.

Las mejoras identificadas en la sección 4 —especialmente la actualización de versiones, la validación server-side del placement, la autenticación y los tests automáticos— constituyen la hoja de ruta natural para una hipotética **Fase 3** orientada a producción y escalabilidad.

■ El código fuente completo del servidor está disponible en:
<https://github.com/Jesus721MLG/server>